

LABSHEET 10

- Step 1. Select a standard dataset such as MNIST, Fashion-MNIST, EMNIST, or CelebA for the classification task.

```
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Dataset loaded successfully.")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("Aryan Agarwal")
print("CU240251800")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 1s 0us/step
Dataset loaded successfully.
Training samples: 60000
Testing samples: 10000
Aryan Agarwal
CU240251800
```

- Step 2. Import the required libraries and load the selected dataset into the development environment.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import models, layers

print("Libraries imported successfully.")

print("Aryan Agarwal")
print("CU240251800")
```

```
Libraries imported successfully.
Aryan Agarwal
CU240251800
```

- Step 3. Perform image preprocessing by resizing (if required), normalization, and splitting the dataset into training, validation, and testing sets

```
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

print("Images normalized and reshaped.")

print("Aryan Agarwal")
print("CU240251800")
```

```
Images normalized and reshaped.
Aryan Agarwal
CU240251800
```

- Step 4. Design a Convolutional Neural Network (CNN) consisting of convolutional layers, pooling layers, activation functions, fully connected layers, and an output layer.

```

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary()

print("Aryan Agarwal")
print("CU240251800")

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 64)	346,176
dense_1 (Dense)	(None, 10)	650

Total params: 347,146 (1.32 MB)
 Trainable params: 347,146 (1.32 MB)
 Non-trainable params: 0 (0.00 B)
 Aryan Agarwal
 CU240251800

- Step 5. Configure the model using an appropriate optimizer, loss function, learning rate, batch size, and number of training epochs.

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Model configured successfully.")

print("Aryan Agarwal")
print("CU240251800")

```

Model configured successfully.
 Aryan Agarwal
 CU240251800

- Step 6. Train the CNN model using the training dataset and monitor the learning process through training and validation accuracy/loss curves.

```

history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

print("Model training completed.")

print("Aryan Agarwal")
print("CU240251800")

```

Epoch 1/5
 844/844 — 30s 33ms/step - accuracy: 0.9304 - loss: 0.2350 - val_accuracy: 0.9777 - val_loss: 0.0833
 Epoch 2/5
 844/844 — 28s 33ms/step - accuracy: 0.9778 - loss: 0.0776 - val_accuracy: 0.9790 - val_loss: 0.0736
 Epoch 3/5

```

844/844 ————— 42s 50ms/step - accuracy: 0.9841 - loss: 0.0534 - val_accuracy: 0.9845 - val_loss: 0.0571
Epoch 4/5
844/844 ————— 37s 44ms/step - accuracy: 0.9883 - loss: 0.0387 - val_accuracy: 0.9867 - val_loss: 0.0514
Epoch 5/5
844/844 ————— 35s 37ms/step - accuracy: 0.9904 - loss: 0.0306 - val_accuracy: 0.9842 - val_loss: 0.0570
Model training completed.
Aryan Agarwal
CU240251800

```

Step 7. Evaluate the trained model on the testing dataset using performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

```

from sklearn.metrics import classification_report, confusion_matrix

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
pred = np.argmax(model.predict(X_test), axis=1)

print("Accuracy:", accuracy)
print(classification_report(y_test, pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))

print("Aryan Agarwal")
print("CU240251800")

```

```

313/313 ————— 3s 11ms/step
Accuracy: 0.9848999977111816

```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	980
1	0.98	1.00	0.99	1135
2	0.98	0.98	0.98	1032
3	0.98	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.98	0.98	892
6	1.00	0.98	0.99	958
7	0.98	0.99	0.99	1028
8	0.99	0.97	0.98	974
9	0.99	0.97	0.98	1009
accuracy			0.98	10000
macro avg	0.99	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

```

Confusion Matrix:
[[ 977  0  0  1  0  1  0  0  1  0]
 [  0 1132  3  0  0  0  0  0  0  0]
 [  3  6 1014  1  0  0  0  5  3  0]
 [  0  0  2 1001  0  2  0  1  4  0]
 [  1  2  1  0 971  0  1  0  0  6]
 [  1  0  1 13  0 876  1  0  0  0]
 [ 10  3  1  0  2  2 940  0  0  0]
 [  0  2  6  1  0  0  0 1018  1  0]
 [  6  1  6  4  3  2  2  2 943  5]
 [  3  6  0  1  8  5  0  8  1 977]]

```

```

Aryan Agarwal
CU240251800

```

Step 8. Perform predictions on unseen images and compare the predicted labels with the actual labels.

```

i = 0
prediction = np.argmax(model.predict(X_test[i:i+1]))

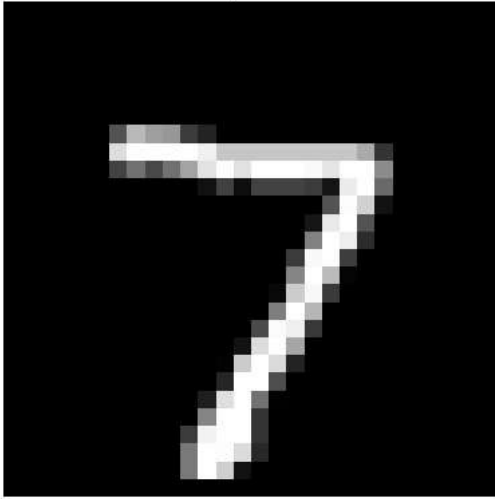
plt.imshow(X_test[i].reshape(28,28), cmap='gray')
plt.title(f"Actual: {y_test[i]}, Predicted: {prediction}")
plt.axis("off")
plt.show()
plt.close()

print("Aryan Agarwal")
print("CU240251800")

```

1/1 — 0s 46ms/step

Actual: 7, Predicted: 7

Aryan Agarwal
CU240251800

- Step 9. Analyze the model's performance, identify possible misclassifications, and discuss factors affecting classification accuracy.

```
wrong = np.sum(pred != y_test)

print("Misclassified images:", wrong)
print("Factors: similar digits, noise, image quality and insufficient training.")

print("Aryan Agarwal")
print("CU240251800")
```

```
Misclassified images: 151
Factors: similar digits, noise, image quality and insufficient training.
Aryan Agarwal
CU240251800
```

- Step 10. Summarize the results and discuss the practical applications of CNN-based image classification in modern computer vision systems.

```
print("CNN successfully classified handwritten digits.")
print("Applications: OCR, handwriting recognition, document processing and biometrics.")

print("Aryan Agarwal")
print("CU240251800")
```

```
CNN successfully classified handwritten digits.
Applications: OCR, handwriting recognition, document processing and biometrics.
Aryan Agarwal
CU240251800
```

Questions:

- Q.1. What is image classification? How does it differ from object detection and image segmentation?

Image classification assigns a single class or category to an entire image. Object detection identifies multiple objects in an image and locates them using bounding boxes along with their class labels. Image segmentation goes further by classifying individual pixels, allowing the exact shape and boundaries of objects to be identified.

✓ Q2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

A Convolutional Neural Network (CNN) consists mainly of convolutional layers, pooling layers, and fully connected layers. The convolutional layers extract features such as edges, shapes, and textures from an image, while pooling layers reduce the spatial size of the feature maps. The extracted features are then passed to fully connected layers for classification. During training, the CNN adjusts its weights using backpropagation to reduce the difference between predicted and actual classes.

✓ Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

Convolutional layers extract important features from images using filters or kernels. Pooling layers reduce the size of feature maps while retaining important information, which decreases computation and helps control overfitting. Fully connected layers combine the extracted features and perform the final classification by producing the output classes.

✓ Q4. Why is image normalization performed before training a deep learning model?

Image normalization scales pixel values to a smaller and consistent range, commonly from 0–255 to 0–1. This makes the input data easier for the neural network to process and helps the model train faster and more stably. It also improves the efficiency of optimization during training.

✓ Q5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

ReLU (Rectified Linear Unit) introduces non-linearity into the CNN and is commonly used in hidden layers, allowing the network to learn complex patterns. Softmax is generally used in the final layer for multi-class classification. It converts the output values into probabilities whose sum is 1, allowing the class with the highest probability to be selected as the prediction.

✓ Q6. What is a Confusion Matrix? How is it used to evaluate classification performance?

A confusion matrix is a table used to compare the actual classes of images with the classes predicted by a classification model. It shows correct predictions and different types of incorrect predictions for each class. It helps identify which classes are being confused and is also used to calculate metrics such as accuracy, precision, recall, and F1-score.

✓ Q7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

Accuracy is the proportion of all correctly classified samples out of the total samples. Precision measures how many of the samples predicted as a particular class are actually members of that class. Recall measures how many of the actual samples of a class were correctly detected by the model. F1-score is the harmonic mean of precision and recall and provides a balanced measure when both are important. For example, in digit classification, if a model predicts many images as digit 5, precision shows how many of those predictions are actually 5, while recall shows how many of all actual 5s were correctly predicted.

✓ Q8. Compare traditional feature-based image classification methods (e.g., SIFT/HOG + SVM) with CNN-based classification.

Traditional methods such as SIFT or HOG with SVM require manually designed features to represent important image information before classification. CNNs automatically learn useful features directly from training images through convolutional layers. Traditional methods can work well with smaller datasets and require less computational power, while CNNs generally require more data and computing resources but can learn more complex visual patterns. Therefore, CNNs are widely used for modern image classification tasks.

✓ Q9. Mention five real-world applications of CNN-based image classification.

CNN-based image classification is used in many real-world applications. Five examples are handwritten digit and character recognition, face recognition, medical image analysis, object and product recognition, and document or optical character recognition (OCR). These applications use CNNs to automatically identify and classify visual patterns in images.

Q10. How can techniques such as Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?

Data augmentation improves model performance by creating variations of training images through operations such as rotation, flipping, cropping, and scaling, which helps the model generalize better. Transfer learning uses a model that has already learned useful features from a large dataset and adapts it to a new classification task, reducing training time and data requirements. Hyperparameter tuning involves selecting suitable values for parameters such as learning rate, batch size, number of layers, and epochs, which can improve training and classification performance.